

**Государственное бюджетное общеобразовательное учреждение города
Москвы «Школа № 192»**

**Учебный тренажёр
“SchoolLinux”.**

Ученик 9В класса Школы № 192
Карасев Антоний Георгиевич

руководитель:
учитель информатики Школы № 192
Бонвеч Елена Алексеевна

Москва, 2026

Оглавление

Оглавление.....	2
Введение.....	3
Цель и задачи.....	5
Методика выполнения работы.....	6
Изучение информации о существующих решениях и терминале Linux.....	6
Сценарий урока и игровой процесс.....	7
Создание базовой системы.....	8
Регистрация.....	8
Взаимодействие пользователей с системой.....	9
Оптимизация создания игровой среды.....	11
Завершение игры.....	11
Апробация и тестирование.....	11
Результаты и обсуждение.....	13
Описание тренажера SchoolLinux.....	14
Установка сервера на компьютер учителя.....	14
Выбор и настройка сценария.....	16
Регистрация учеников.....	18
Начало игры.....	18
Течение игры.....	19
Завершение.....	21
Список использованной литературы.....	22

Введение

В условиях импортозамещения российские школы активно переходят с операционной системы Windows на отечественные дистрибутивы на базе ядра Linux. В Москве повсеместно внедряется операционная система МЭШ (МОС) — дистрибутив на базе РОСА Linux, разработанный Департаментом информационных технологий Москвы. В других регионах РФ используются ALT Linux, РЕД ОС Образовательная редакция и Astra Linux. К началу 2024/2025 учебного года доля отечественного ПО в школах достигла 60 процентов [1].

Этот переход создаёт потенциальную проблему: большинство школьников умеет работать исключительно с графическим интерфейсом, тогда как даже базовое знакомство с терминалом Linux расширяет их цифровую грамотность. Терминал не является обязательным инструментом для повседневной работы, но понимание командной строки развивает системное мышление, учит работе с файловой системой на низком уровне и формирует навыки, востребованные при дальнейшем изучении программирования и информационных технологий. При этом в рамках школьной программы терминалу практически не уделяется внимания, а существующие методические материалы носят формальный характер без элементов вовлечения.

Настоящий проект создан для того, чтобы превратить изучение терминала из скучного перечисления команд в увлекательный процесс через интерактивную веб-платформу. Платформа реализована в формате игрового практикума, где учитель централизованно управляет занятием, а ученики подключаются к системе через браузер без установки дополнительного программного обеспечения. Ключевые возможности включают модульную архитектуру игровых сценариев, автоматическое развёртывание заданий на компьютерах учеников по протоколу SSH, мониторинг прогресса в реальном времени и поддержку нескольких режимов взаимодействия (поиск информации и самостоятельное создание файловых структур).

Такой подход соответствует современным принципам обучения: информация, полученная через интерактив и самостоятельное открытие, запоминается значительно эффективнее.

Среди существующих образовательных решений можно выделить англоязычный тренажёр OverTheWire (Bandit) [2], однако он не адаптирован под российскую школьную программу, не поддерживают русскоязычный интерфейс и не интегрируются в учебный процесс как инструмент учителя. Критически важно, что подобные платформы не позволяют педагогу отслеживать прогресс группы в реальном времени, выставять оценки и контролировать выполнение задания централизованно, это может вызывать сложности с их применением на уроках информатики в условиях массового перехода школ на отечественные ОС.

Цель и задачи

Целью настоящего проекта является разработка веб-приложения SchoolLinux, позволяющего организовать игровой урок по освоению базовых и продвинутых команд терминала Linux в условиях школьного класса без необходимости специальной предварительной настройки оборудования.

Основные задачи проекта:

Аналитическая задача: изучить существующие подходы к обучению командной строке Linux, выявить их недостатки для применения в школьной среде и определить минимально необходимый набор команд для освоения (навигация по файловой системе, поиск файлов, работа с текстом, распаковка архивов).

Архитектурная задача: спроектировать клиент-серверную архитектуру приложения с модульным бэкендом и отдельным интерфейсом, где серверная часть размещается на компьютере учителя, а клиентская часть доступна ученикам через браузер без установки дополнительного ПО.

Реализация механизма развёртывания и игрового движка: создать систему автоматической генерации и развёртывания игровых сред на компьютерах учеников по протоколу SSH, основанную на подключаемых модулях с настраиваемыми параметрами сложности и режимами работы.

Реализация учебного процесса: разработать интерфейс для учителя с возможностью выбора игрового сценария, просмотра списка подключённых учеников, управления занятием в реальном времени и автоматического экспорта результатов в формат CSV для выставления оценок.

Апробация и тестирование: провести пилотное внедрение приложения на уроках информатики, обеспечить автоматизированное тестирование серверной логики и непрерывную интеграцию для стабильности работы в условиях школьной сети.

Методика выполнения работы

Для реализации проекта был выбран современный универсальный язык программирования Python 3.12 [3] в сочетании с асинхронным веб-фреймворком FastAPI [4] для серверной части, языком TypeScript [5] и библиотекой React [6] для клиентского интерфейса, библиотекой Axios [7] для обмена данными с бэкендом по REST API, а также сборщиком Vite [8] для оптимизированной компиляции фронтенда из TypeScript в JavaScript [9].

Для организации обмена данными в реальном времени применяется протокол Socket.IO [10] поверх ASGI, а для удалённого управления учебными машинами — библиотека Paramiko [11] (SSHv2).

Состояние системы сериализуется в типизированный JSON-формат с валидацией через Pydantic [12], что исключает риски повреждения или подделки данных, упрощает отладку.

Для обеспечения стабильности внедрён фреймворк pytest и настроены пайплайны непрерывной интеграции для автоматической проверки кода и сборки релизных архивов.

Выбор FastAPI обусловлен поддержкой асинхронных запросов и автоматической генерацией документации, что упрощает отладку и развитие проекта.

Изучение информации о существующих решениях и терминале Linux

На подготовительном этапе проведён анализ педагогических подходов к обучению командной строке. Изучен англоязычный тренажёр OverTheWire (Bandit) [2]. Выявлен ключевой недостаток аналогов: отсутствие интеграции в школьный учебный процесс — невозможность централизованного управления заданиями учителем, гибкой настройки сложности и автоматической оценки результатов группы учеников. Определён минимально необходимый набор команд терминала для освоения на уроке: навигация, просмотр содержимого, работа с архивами, поиск по содержимому и именам. Совместно с учителем

разработан сценарий урока и правила игрового процесса, учитывающие централизованный запуск раундов, автоматическое развёртывание сред и два базовых режима взаимодействия: поиск информации и самостоятельное создание файловых структур по заданию преподавателя.

Сценарий урока и игровой процесс

Игровой процесс реализуется через систему подключаемых сценариев, которые преподаватель выбирает перед стартом раунда на панели управления. Введено понятие “клада” - текстовой строки в файле, которая начинается с префикса “klad:” (пример “клада”: “klad:something”). “Клады” можно искать или прятать в зависимости от выбранного сценария игры.

Игровые механики (соревнование, мгновенная обратная связь, визуализация прогресса) соответствуют принципам геймификации, повышающим вовлечённость учащихся.

В режиме поиска система генерирует индивидуальное файловое дерево для каждого ученика. Параметры сложности настраиваются вручную: глубина вложенности папок, общее количество файлов и директорий, доля сжатых архивов, а также количество скрытых целей. Цель ученика — найти “клады” среди случайно сгенерированных данных, используя команды терминала.

В режиме создания структуры ученики получают задание самостоятельно сформировать файловое дерево с заданными параметрами и расположением “кладов”, после чего система проверяет корректность выполнения.

Преподаватель отслеживает прогресс в реальном времени через веб-интерфейс и может прервать игру в любой момент. По окончании раунда система автоматически подсчитывает баллы и формирует отчёт.

Создание базовой системы

Для организации работы приложения используется модульная клиент-серверная архитектура. Бэкенд разделён на логические компоненты: маршрутизаторы API, бизнес-логика, схемы данных, менеджер сокет-соединений, игровые плагины. Каждый игровой сценарий реализуется как отдельный Python-файл, наследующий базовый класс игры, что позволяет добавлять новые режимы без изменения ядра системы.

Для удалённого подключения к учебным машинам применяется протокол SSH с параметризованной аутентификацией.

Состояние приложения хранится в типизированном JSON-файле, который автоматически восстанавливается при перезапуске сервера и обеспечивает прозрачность данных и преемственность работы.

Ролевая модель основана на анализе IP-адресов подключений, при этом локальный адрес компьютера учителя автоматически определяет роль администратора. Глобальная машина состояний системы использует явные статусы: idle (ожидание), init (развёртывание), running (активная игра).

Регистрация

После перехода на адрес сервера система предлагает ученику ввести свои фамилию и имя для привязки к учётной записи и последующего формирования отчёта, см. рис. 1. При нехватке компьютеров допускается регистрация команд через указание нескольких имён.

➔ Регистрация

💻 Номер компьютера
► Рекомендуемый формат
Номер компьютера

👤 Полное имя
► Рекомендуемый формат
Полное имя

➔ Подключиться

Вы можете быть дисквалифицированы за грубое несоответствие данных формату/
некорректные данные

Рисунок 1. Форма регистрации ученика

Данные о подключении ученика фиксируются автоматически: IP-адрес учебной машины извлекается из метаданных сетевого соединения HTTP и WebSocket.

После успешной регистрации ученик отображается на панели преподавателя. Дальнейшая идентификация и маршрутизация игровых событий производятся строго по уникальному IP-адресу клиента, что исключает необходимость ручной настройки или ввода паролей на стороне учеников.

Взаимодействие пользователей с системой

Пользователи взаимодействуют с системой через современный веб-интерфейс, разделённый на два независимых экрана: панель преподавателя и рабочее пространство ученика.

Преподавателю предоставляется централизованная панель управления с возможностью выбора игры, настройки параметров сложности, запуска и остановки раунда, а также мониторинга активности всех подключённых устройств в реальном времени. Пример панели управления приведен рис. 2.

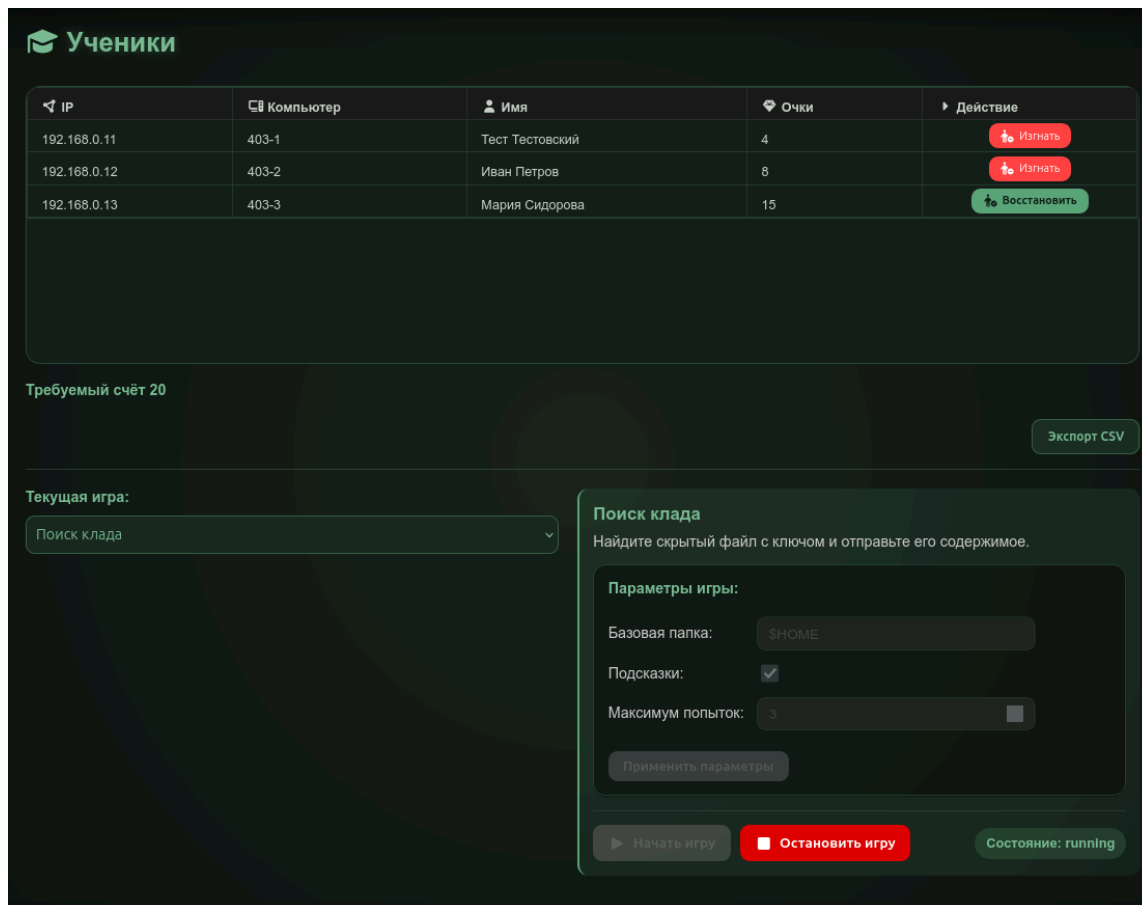


Рисунок 2. Панель управления учителя

Обмен данными между компьютерами осуществляется через событийную модель Socket.IO [9], которая мгновенно транслирует изменения статусов, обновление списков пользователей и результаты проверок.

Ученики видят форму отправки “кладов” или кнопку проверки состояния файловой структуры на их компьютере (система ищет “клады”), текущее состояние игры и уведомления о статусе.

Все действия проверяются на стороне сервера с учётом роли пользователя и текущего глобального состояния раунда.

Оптимизация создания игровой среды

Игровая среда разворачивается на компьютерах учеников через защищённое SSH-соединение с использованием методов плагинов. Вместо последовательной отправки отдельных команд система может сформировать оптимизированный сценарий развёртывания, который передаётся и выполняется на удалённой машине за одну сессию. Встроенные игровые сценарии по умолчанию используют это, что минимизирует сетевые задержки и снижает нагрузку на канал связи.

Архитектура плагинов позволяет изолировать логику генерации от ядра системы: каждый плагин реализует методы установки, проверки и очистки, что гарантирует предсказуемое поведение системы даже при нестандартных конфигурациях файловых систем на учебных машинах.

Завершение игры

После получения команды на завершение игры сервер блокирует приём новых ответов от учеников, переходит в режим простоя, запуская процедуру очистки. На компьютерах учеников автоматически выполняются команды удаления созданных во время раунда файлов и директорий, что возвращает систему в исходное состояние без ручного вмешательства.

Интерфейс учителя формирует итоговую таблицу с результатами и инициирует её автоматическую загрузку через браузер в формате CSV на компьютер учителя. Данные о регистрациях сохраняются в файле состояния приложения, что позволяет проводить несколько раундов подряд без повторного ввода данных учениками даже после перезапуска сервера системы.

Апробация и тестирование

Тестирование актуальной реализации системы проведено при содействии учителей информатики в компьютерном классе Школы №192 на уроках информатики в 9В и 10В классах. Использовался дистрибутив МОС 12 [13]. В

процессе апробации особое внимание уделялось стабильности SSH-подключений при одновременном развёртывании сред на 25–30 машинах, корректности работы плагинной системы и синхронизации интерфейсов через Socket.IO.

Для обеспечения качества кода внедрён набор автоматизированных тестов серверной логики на базе `pytest`, а также настроены пайплайны непрерывной интеграции, которые проверяют сборку фронтенда и запуск тестов при каждом обновлении кода. Критические сценарии потери состояния и сетевые ошибки были обнаружены и устранены до финального релиза.

Результаты и обсуждение

Результатом проекта является готовая к эксплуатации учебная платформа для учителей информатики, адаптированная для использования в компьютерных классах российских школ. Реализация поддерживает настраиваемые режимы игры, модульную архитектуру, типизированное хранение состояния и оптимизированное обновление данных в реальном времени.

Получены положительные отзывы от преподавателей и учеников, отмечена высокая скорость развёртывания сред и удобство мониторинга прогресса в реальном времени.

Разница во времени поиска “кладов” при использовании базовых и продвинутых команд терминала наглядно демонстрирует эффективность использованного подхода, показывает ученикам практическую ценность изучения командной строки (в ходе тестирования зафиксировано, что ученики, применявшие команды `grep` и `find`, завершали поиск в среднем на 30–40% быстрее).

Дальнейшее развитие проекта предполагает внедрение встроенного курса по Linux, существует возможность использовать написанный код для проведения практических работ с использованием других технологий, например, таких, как система контроля версий Git.

Исходный код проекта представлен в GitHub репозитории:
<https://github.com/A2020GK/SchoolLinux>

Актуальная версия полного описания проекта находится по ссылке:
https://docs.google.com/document/d/1qG4EtB_5OPL7i3gDynXxIVqjF_16OSj3mzS_WsxPnY2c/edit?usp=sharing

Актуальная версия презентации проекта находится по ссылке:
<https://docs.google.com/presentation/d/1O3jT2FiviXqyDknXagMJK1GyMgmgXG2-cabbvkh5Jz0/edit?usp=sharing>

Описание тренажера SchoolLinux.

Тренажёр SchoolLinux представляет собой современное решение для интеграции отработки практических навыков работы с терминалом Linux в школьный образовательный процесс.

Продукт учитывает специфику российской школьной среды, требования к централизованному управлению и необходимости минимальной предварительной настройки оборудования.

Для работы системы требуется Linux на сервере и клиентах, открытый порт 22 на учебных машинах, одни и те же имена пользователей и пароли на разных компьютерах для SSH-доступа по одним и тем же учётным данным и стабильное сетевое подключение в рамках локальной сети.

Ключевыми характеристиками системы являются модульная плагиновая архитектура, замена бинарной сериализации на типизированный JSON с валидацией, разделение интерфейсов преподавателя и ученика, внедрение событийной синхронизации и автоматизация сборки релизных пакетов.

Генератор игровых сред обеспечивает баланс между временем инициализации и сложностью заданий, различные типы заданий расширяют методический потенциал системы.

Установка сервера на компьютер учителя

Для установки и запуска серверной части системы на компьютере учителя доступны два варианта.

Вариант 1: Готовый релизный архив (рекомендуется для быстрого развёртывания)

1. Перейдите на страницу GitHub Releases проекта и скачайте актуальный архив SchoolLinux-версия.tar.gz.

2. Распакуйте архив на целевой машине учителя.
3. Измените файл `.env`, внося туда учётные данные пользователя, который есть на всех компьютерах (переменные `SSH_USER` и `SSH_PASSWORD`).
4. Запустите скрипт установки командой `./install.sh`
5. Запустите сервер командой `./run.sh`. Интерфейс системы будет доступен на порту 8000. Локальный и сетевой адреса интерфейса занятия отобразятся в терминале в выводе этой команды.

Вариант 2: Ручная установка из исходного кода

1. Клонировать репозиторий командой `git clone https://github.com/A2020GK/SchoolLinux` и перейдите в папку проекта.
2. Создайте и активируйте виртуальное окружение Python: `python -m venv .venv`, `source .venv/bin/activate`.
3. Установите зависимости: `pip install -r requirements.txt`.
4. Создайте файл `.env` в корне проекта на основе `.env.example` и укажите `SSH_USER`, `SSH_PASSWORD`, а также при необходимости `ALLOW_IP_OVERRIDE=false` для локального тестирования системы.
5. Установите зависимости фронтенда: `npm install`.
6. Запустите серверную часть: `fastapi run --host 0.0.0.0`.
7. В отдельном терминале запустите клиентскую часть в режиме разработки: `npm run vite --host 0.0.0.0 --port 4000`. Адрес, на котором будет доступна система отобразится в выводе этой команды.

Локальный адрес (`localhost`) используется учителем для просмотра интерфейса, сетевой адрес может быть написан на доске/сообщён ученикам, чтобы они могли открыть интерфейс системы в браузере и зарегистрироваться.

Актуальная версия инструкции по установке и запуску сервера системы размещена в файле `README.md` в репозитории тренажера на GitHub (<https://github.com/A2020GK/SchoolLinux#2-руководство-по-установке>)

Выбор и настройка сценария

Для начала раунда преподаватель выбирает нужный игровой сценарий на панели управления, настраивает параметры сложности и нажимает кнопку запуска.

Встроенные сценарии:

1. Прятание “кладов”: Ученики создают файловое дерево в каталоге \$HOME/Game, проверка осуществляется через анализ содержимого папки \$HOME/Game.

Параметры:

1. Целевое количество кладов - Сколько файлов с “кладами” необходимо спрятать в дереве.
2. Минимальное количество файлов, которые не являются кладом
3. Разрешать ли пустые файлы
4. Разрешать ли оставлять кладов непосредственно в \$HOME/Game, или ученики должны создать подкаталоги
5. Количество каталогов, которые ученики должны создать в папке \$HOME/Game
6. Количество каталогов, которые должны содержать в себе минимальный размер файлового дерева в глубину
7. Минимальный размер файлового дерева в глубину

Пункты 6 и 7 нужны для отработки навыков создания сложных файловых структур. Пример работы этих параметров: при минимальном размере 3 и количестве каталогов 1, путь к валидному каталогу должен выглядеть как “\$HOME/Game/folder1/folder2/folder3”. Если количество

каталогов равно 2, то система будет искать каталоги с путями вида:

- \$HOME/Game/folder1/folder2/folder3
- \$HOME/Game/folder2/folder3/folder4

Максимальный балл - 10, за нарушения различных критериев снимается различное количество баллов, что обеспечивает сбалансированную оценку для различных параметров.

2. Поиск “кладов”: Система прячет “клады” случайным образом для каждого ученика в файлах в дереве в каталоге \$HOME/Game, найденные “клады” сдаются в систему целиком (“klad:” и последующее содержимое строки)

Параметры:

1. Количество кладов - сколько всего файлов с “кладами” должно быть в создаваемом файловом дереве
2. Количество файлов - сколько всего файлов должно быть в создаваемом файловом дереве (файлы с “кладами” включены в это число).
3. Количество строк в файлах - сколько строк со случайными данными должно генерироваться в файлах (файлы с “кладами” тоже содержат случайные данные)
4. Максимальная глубина дерева - насколько глубоко (\$HOME/Game/folder1/folder2/.../foldern, где n - значение параметра) может сгенерироваться дерево.
5. Процент .gz-файлов - какая доля файлов должна быть сжаты с использованием gzip. Это необходимо для отработки работы с архивами, при установке значения 0 файлы не сжимаются.

Регистрация учеников

Ученики должны войти в систему на своих компьютерах, используя учётные данные пользователя, который прописан в файле `.env` (см. инструкцию по установке). Затем ученики должны открыть в браузере веб-страницу, расположенную по адресу компьютера учителя в локальной сети (отображается в выводе команды запуска), написать имя, фамилию, номер своего компьютера, нажать на кнопку регистрации. При успешной регистрации ученик появляется в таблице в интерфейсе учителя. В случае, если доступ по SSH с указанными в файле `.env` учётными данными невозможен, регистрация не фиксируется.

Начало игры

Серверная часть переходит в состояние инициализации и разворачивает индивидуальные игровые среды на всех зарегистрированных компьютерах через SSH. Игра начинается после успешной подготовки сред на всех подключённых компьютерах, что гарантирует равные условия для участников. Статус раунда и готовность систем транслируются в интерфейс учителя и учеников в реальном времени, см. рис. 3.

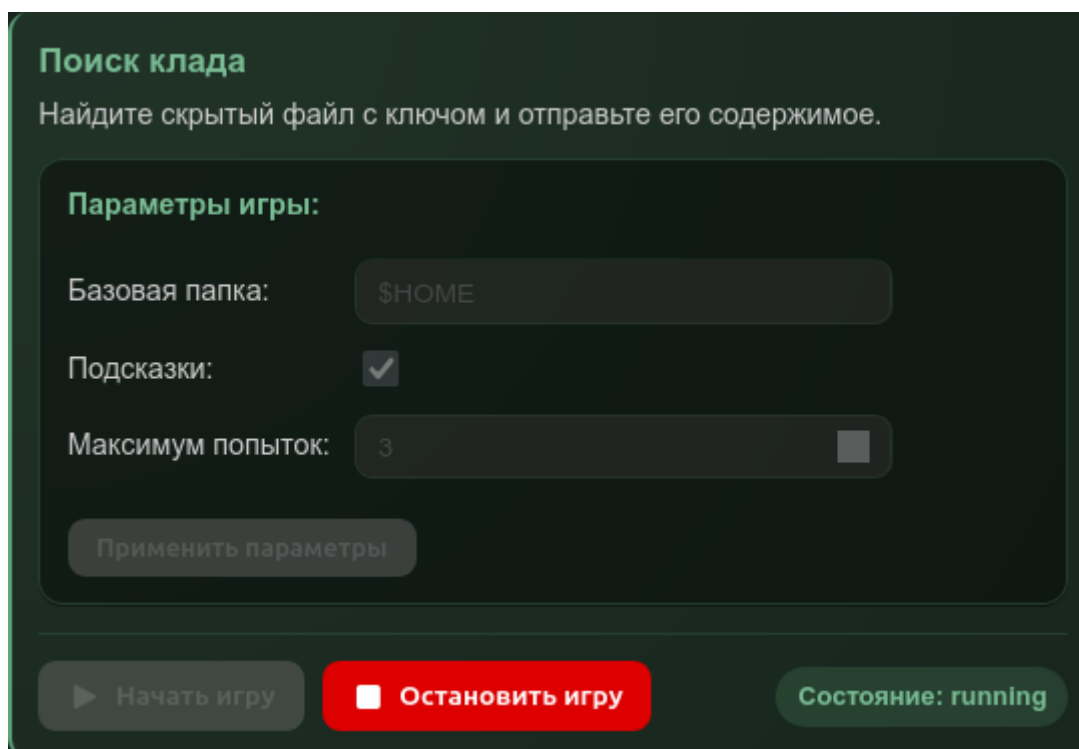


Рисунок 3. Статус игры на панели учителя

Течение игры

Во время активной фазы в интерфейсе преподавателя отображается обновляющаяся в реальном времени таблица с текущими баллами, статусом подключения и историей попыток каждого ученика, рис. 4.

Ученики				
IP	Компьютер	Имя	Очки	Действие
192.168.0.11	403-1	Тест Тестовский	4	Игнать
192.168.0.12	403-2	Иван Петров	8	Игнать
192.168.0.13	403-3	Мария Сидорова	15	Восстановить
Требуемый счёт 20				
				Экспорт CSV

Рисунок 4. Таблица с данными учеников

Участники работают в терминале Linux, выполняя команды навигации, и работы с файлами в соответствии с выбранным режимом. Найденные значения вводятся в веб-форму ученика или отправляются автоматически в зависимости от настроек плагина, см. рис 5.

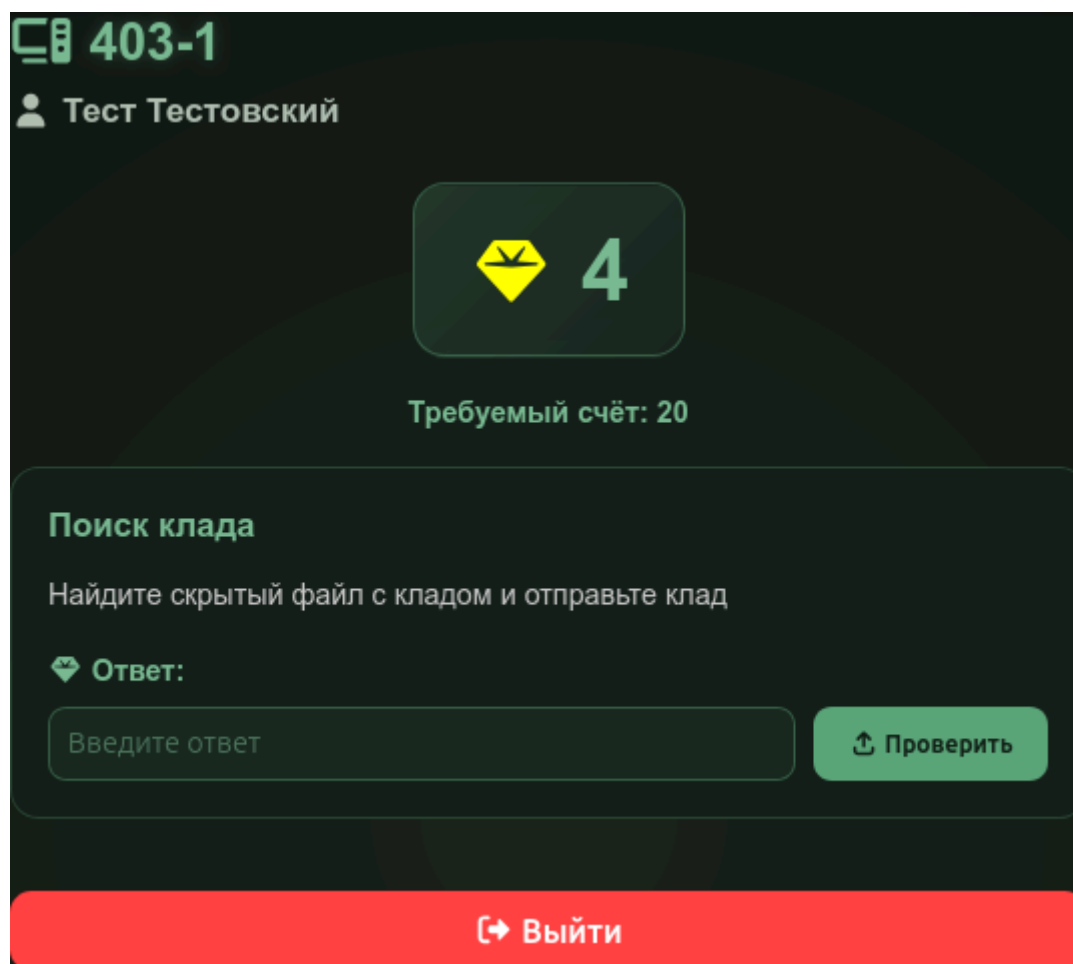


Рисунок 5. Форма ввода “клада” ученика

Сервер валидирует ответ, начисляет баллы и мгновенно обновляет данные через Socket.IO. Каждая цель может быть засчитана только один раз. Преподаватель может в любой момент просмотреть детализацию, отправить предупреждение или принудительно завершить раунд.

Завершение

Для завершения раунда преподаватель нажимает кнопку остановки игры. Сервер блокирует приём новых ответов, переводит систему в состояние ожидания и запускает процедуру очистки на удалённых машинах через метод очистки каждого плагина. Встроенные игровые сценарии обеспечивают автоматическое удаление всех созданных во время игры файлов и директорий.

Система формирует итоговый отчёт в формате CSV, идентичный таблице на панели преподавателя, и инициирует его сохранение в браузере. Путь сохранения определяется настройками браузера.

Данные о регистрациях сохраняются, что позволяет начать новый раунд без повторного ввода имён.

Все технические вопросы и отчёты об ошибках рекомендуется направлять в раздел Issues на GitHub.

Список использованной литературы

1. Школы и вузы к сентябрю успели на 60% перейти на отечественное ПО / Редакция сайта ТАСС // URL: <https://tass.ru/obschestvo/21721377>, (дата последнего обращения: 31.01.2026)
2. OverTheWire - Bandit // URL: <https://overthewire.org/wargames/bandit/> (дата последнего обращения: 10.11.2025)
3. Документация по языку программирования Python версии 3.12.2 // URL: <https://docs.python.org/3/index.html> (дата последнего обращения: 10.11.2025)
4. Документация веб-фреймворка FastAPI // URL: <https://fastapi.tiangolo.com/learn/> (дата последнего обращения: 10.11.2025)
5. Документация по языку программирования TypeScript // URL: <https://www.typescriptlang.org/docs/> (дата последнего обращения: 09.11.2025)
6. Документация по библиотеке React // URL: <https://react.dev/learn> (дата последнего обращения: 03.10.2024)
7. Документация по библиотеке Axios // URL: <https://github.com/axios/axios> (дата последнего обращения: 10.11.2025)
8. Документация по среде сборки JavaScript Vite // URL: <https://vite.dev/guide/> (дата последнего обращения: 10.11.2025)
9. Современный Онлайн-учебник по языку JavaScript / Илья Кантор 2007—2026 // URL: <https://learn.javascript.ru/> (дата последнего обращения: 19.02.2024)
10. Документация по библиотеке Socket.IO // URL: <https://socket.io/docs/v4/> (дата последнего обращения: 10.11.2025)
11. Документация по библиотеке Paramiko // URL: <https://docs.paramiko.org/en/stable/> (дата последнего обращения: 09.11.2025)

12. Документация по библиотеке Pydantic // URL: <https://pydantic.dev/docs/validation/latest/> (дата последнего обращения: 09.11.2025)
13. Официальный аккаунт группы разработки операционной системы МОС 12 на МосХаб // URL: <https://hub.mos.ru/mos> (дата последнего обращения: 09.11.2025)